

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-abe5186c17bf57bdb.jsonl`  
- SHA-256: `d2f4728917e74fe5596f51c29090fd94deb683d8e86dd399bd5fb0a5de1fcb2b`  
- Source modified: `2026-06-21T13:57:21+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_a2b97cff-dcd`  
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`
```

Transcript

1. user (2026-06-21T13:52:09.110Z)

Research freezing a Python app into a standalone binary/installer: PyInstaller, Nuitka, cx_Freeze, Briefcase. Be HONEST about feasibility for a torch + opencv + numpy + fastapi + uvicorn app: known pitfalls (torch/opencv dynamic libs + hidden imports, huge size, model files, multiprocessing/uvicorn workers, code-signing/notarization on Win+Mac, antivirus false positives). License check: PyInstaller (GPL w/ exception?), Nuitka (Apache-2/commercial?). Verdict: viable or not, and for which TIER (Gemini-only no-torch tier vs full local-model tier). Use web search.

Return the RESEARCH schema. Context about our app:

PROJECT: a local, AI-powered badminton/racket-sports HIGHLIGHT INDEXER + rally detector.
Engine repo (PRIMARY): C:/Users/avidu/Projects/badminton-highlight-indexer (Python, FastAPI + SQLite + ffmpeg + GPU CV/AI; paid Gemini = cloud AI).
Sibling repos: C:/Users/avidu/Projects/khelsutra (the web/ React SPA + site/ marketing), C:/Users/avidu/Projects/rally-annotator (VLC plugin, MIT), C:/Users/avidu/Projects/sports-data-collector, C:/Users/avidu/Projects/sports-obsreport.

GOAL of the plan we are building: bake the engine into a DOWNLOADABLE, "batteries-included" Python APP that a user can download and run WITHOUT sourcing the repo (no git clone / build-from-source). It must spin up a local WEB SERVICE that runs (a) INFERENCE (video -> rallies -> highlight reel), (b) TRAINING (BYO model on their own labelled data), and (c) RALLY ANNOTATION if needed -- runnable BOTH locally and against CLOUD services. Preserve every already-completed end-to-end CUJ.

KEY FACTS the orchestrator already verified:

```
- pyproject.toml: hatchling build, namespace package (no backend/__init__.py),  
packages=["backend","training"], console script: indexer = backend.main:main. requires-python  
>=3.10.  
- torch/torchvision are DELIBERATELY NOT in dependencies -- their CUDA/CPU wheels need --index-url (PEP621 cannot express). Optional extras: gpu(empty, docs-only), auth(PyJWT),  
reporting(git+ssh obsreport), storage-s3(boto3), storage-gcs(google-cloud-storage). google-genai is a hard dep.  
- Server today: python -m backend.main --server --port 8000 . Worker: python -m backend.worker {infer|train} . CLI single-shot: python -m backend.main --video-path ... --segmenter {gemini|yolo_hybrid|wasb_hybrid|motion}.  
- THE HARD PACKAGING PROBLEM: native WASB shuttle model runs in a SEPARATE Python env (conda, torch 1.11+cu113) while the main engine uses torch 2.6+cu124. ffmpeg/ffprobe are system deps. Model weights (wasb_badminton_best.pth.tar) live in a GCS bucket.  
- Decoupled-compute SEAM already built (M1-M8 merged, default-OFF): deployment.profile + compute_target on AppConfig; backend/config/presets.py (PROFILE_PRESETS: truly-local / cloud-serving / cpu-mock); backend/storage/* (local/gcs/s3/gdrive StorageBackend registry); backend/compute/* (ComputeBackend registry, CloudRunCompute dispatches a Cloud Run Job); backend/config/startup.py per-profile checks; backend/billing.py cost ledger.  
- The web SPA lives in a DIFFERENT repo (khelsutra/web, Vite/React/TS/Tailwind, i18n en+kn+hi).
```

The engine has a likely-DEAD backend/api/static. Bundling the UI is a cross-repo concern.

- Licensing guardrail (NON-NEGOTIABLE): every dependency -- code, data, AND weights -- must be MIT/Apache-2.0/BSD only. Paid LLMs = inference/serving only, never training data. Any NEW packaging dependency must also be license-clean.
- A prior docs/DESKTOP_APP_PLAN.md exists but was SUPERSEDED by owner decision D10: deliver a "self-hosted webserver + UI the user spins up" (NOT a desktop app).

khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md is the live appliance design.

Always ground claims in file:line or a doc path. Read the actual files; do not assume.

2. assistant (2026-06-21T13:52:12.619Z)

I'll research freezing this Python app into a standalone binary. Let me first ground myself in the actual repo facts, then do web research on the freezing tools.

3. user (2026-06-21T13:52:14.129Z)

```
1      # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2      #
3      # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4      # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5      # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6      # auto-detection would otherwise miss the un-__init__'d `backend` root.
7      #
8      # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9      # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10     # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11     # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12     [build-system]
13     requires = ["hatchling"]
14     build-backend = "hatchling.build"
15
16     [project]
17     name = "badminton-highlight-indexer"
18     version = "0.1.0"
19     description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20     readme = "README.md"
21     requires-python = ">=3.10"
22     license = { text = "MIT" }
23     authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24     keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26     # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27     dependencies = [
28         "fastapi>=0.111.0",
29         "uvicorn>=0.30.1",
30         "opencv-python>=4.9.0.80",
31         "numpy>=1.26.4",
32         "pydantic>=2.7.1",
33         "jinja2>=3.1.4",
34         "python-dotenv>=1.0.0",
35         "google-genai>=2.4.0",
36         "supervision>=0.21.0,<0.30.0",
37     ]
38
39     [project.optional-dependencies]
40     # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41     dev = [
42         "pytest",
43         "pytest-cov",
44         "httpx",          # fastapi TestClient transport
45         "ruff",
46         "mypy",
47     ]
48
```

```

49 # Per-video observability reports (SOFT dependency: the pipeline degrades to a
50 # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
51 # fetched over SSH; if you lack access, simply omit this group ? reporting
52 # tests skip via pytest.importorskip. For local dev against a sibling checkout:
53 # pip install -e ../sports-obsreport --no-build-isolation
54 reporting = [
55     "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
56 ]
57
58 # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
59 # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
60 # express, so they are NOT declarable here. Install them out-of-band instead:
61 # # CUDA 12.4 box:
62 # pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
63 # # CPU-only box (also what CI uses):
64 # pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
65 gpu = []
66
67 # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
68 # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
69 # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
70 # DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
71 # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
backend/storage/gdrive.py.
72 storage-s3 = ["boto3>=1.34"]
73 storage-gcs = ["google-cloud-storage>=2.16"]
74
75 [project.scripts]
76 # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
77 indexer = "backend.main:main"
78
79 [tool.hatch.metadata]
80 # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
81 # reference*); hatchling requires opting in to allow those in metadata.
82 allow-direct-references = true
83
84 [tool.hatch.build.targets.wheel]
85 # `backend` is a namespace package (no __init__.py); list both import roots
86 # explicitly so an editable/wheel install exposes `import backend` and
87 # `import training` regardless of cwd.
88 packages = ["backend", "training"]
89

```

4. assistant (2026-06-21T13:52:18.139Z)

Now let me research the freezing tools. I'll fan out web searches and read the relevant repo/docs files in parallel.

5. user (2026-06-21T13:52:20.127Z)

docs\DESKTOP_APP_PLAN.md

6. user (2026-06-21T13:52:25.561Z)

Web search results for query: "PyInstaller PyTorch torch opencv bundling pitfalls hidden imports large size 2025"

Links: [{"title": "Process of making EXE file using python | by Ayub Alam | Medium", "url": "https://ayubalam.medium.com/process-of-making-exe-file-using-python-ea8ef79a1405"}, {"title": "When Things Go Wrong ? PyInstaller 6.21.0 documentation", "url": "https://pyinstaller.org/en/stable/when-things-go-wrong.html"}, {"title": "Pyinstaller and Pytorch · pyinstaller · Discussion #7621", "url": "https://"}]

```
github.com/pyinstaller/pyinstaller/discussions/7621"}, {"title": "Simplifying Deployment: Using PyInstaller with PyTorch ? codegenes.net", "url": "https://www.codegenes.net/blog/pyinstaller-pytorch/"}, {"title": "Pyinstaller problem with a module that uses torch and pytorch_lightning. · pyinstaller · Discussion #7944", "url": "https://github.com/orgs/pyinstaller/discussions/7944"}, {"title": "Changelog for PyInstaller ? PyInstaller 6.3.0 documentation", "url": "https://pyinstaller.org/en/v6.3.0/CHANGES.html"}, {"title": "Unable to import torch in an exe · Issue #8348 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/8348"}, {"title": "pyinstaller package size for spacy has significantly increased · Issue #4548 · explosion/spaCy", "url": "https://github.com/explosion/spaCy/issues/4548"}]
```

I've found relevant search results for your query about PyInstaller bundling with PyTorch and OpenCV. Here are the key findings:

Main Issues and Pitfalls

****Large Executable Sizes:****

PyInstaller executables using PyTorch, OpenCV, and PyQt5 libraries can reach approximately 250MB or more.

****Hidden Imports and Metadata Issues:****

PyTorch requires metadata to be collected during bundling, which can be done by adding `--copy-metadata torch` to your PyInstaller command. Additionally, specific hidden imports like `torch._C._jit` and `torch._C._nvrtc` may need to be explicitly included.

****File Duplication Problems:****

When building onefile applications with torch, warnings about file duplication can occur, such as "WARNING: file already exists but should not" for files like `torch/_C.cp39-win_amd64.pyd`.

Size Optimization Strategies

To reduce executable size, several approaches are recommended:

1. Using the `--exclude-module` option to exclude unnecessary modules, such as `torch.cuda` if CUDA is not needed
2. Excluding modules like `torchvision`, `matplotlib`, `pandas`, and `scipy` when not needed, and using `opencv-python-headless` instead of the full OpenCV package
3. Using tools like UPX for compression or exporting models to more efficient formats like ONNX

The search results provide practical guidance on configuration options and common pitfalls users encounter when bundling machine learning applications with PyInstaller.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

7. user (2026-06-21T13:52:29.491Z)

Web search results for query: "Nuitka commercial license vs open source Apache PyTorch standalone binary 2025"

Links: [{"title": "Nuitka | pydevtools", "url": "https://pydevtools.com/handbook/reference/nuitka/"}, {"title": "GitHub - Nuitka/Nuitka: Nuitka is a Python compiler written in Python. It's fully compatible with Python 2.6, 2.7, 3.4-3.13. You feed it your Python app, it does a lot of clever things, and spits out an executable or extension module. · GitHub", "url": "https://github.com/Nuitka/Nuitka"}, {"title": "License of Nuitka Commercial ? Nuitka the Python Compiler", "url": "https://nuitka.net/doc/commercial-license.html"}, {"title": "FAQ using Nuitka in commercial products · Issue #245 · Nuitka/Nuitka", "url": "https://github.com/Nuitka/Nuitka/issues/245"}, {"title": "Nuitka Commercial ? Nuitka the Python Compiler", "url": "https://nuitka.net/doc/commercial.html"}, {"title": "Nuitka ·"}]

```
PyPI", "url": "https://pypi.org/project/Nuitka/"), {"title": "Create standalone nuitka deployment  
with only some parts of PyTorch · Issue #2218 ·  
Nuitka/Nuitka", "url": "https://github.com/Nuitka/Nuitka/issues/2218"}, {"title": "Nuitka -  
Wikipedia", "url": "https://en.wikipedia.org/wiki/Nuitka"}]
```

Based on the search results, here's what I found about Nuitka's commercial license compared to its open source offering, with some context about PyTorch binary compilation:

Nuitka License Models

****Open Source Version:****

Nuitka is AGPL-3.0+ with a runtime exception for compiled output, so the license does not carry over to the applications it produces. This means you can use the free version to create commercial applications, but the Nuitka source code itself must comply with AGPL-3.0 requirements.

****Commercial Version:****

When you buy Nuitka Commercial, parts of the code - mostly the plugins that implement the commercial features - are under a license that forbids you to distribute the Nuitka Commercial source code. However, you are free to distribute compiled binaries of your software without any restrictions.

Changed versions, or versions of changes sent to the copyright holder Kay Hayen are automatically licensed to him under the Apache License, Version 2.0 to allow opening and closing of the code by his discretion.

PyTorch Support

The search results indicate that recent releases added built-in package-specific rules for PyTorch, TensorFlow, scikit-learn, and SciPy. The rules prune unused operator kernels and skip compiling TensorFlow's generated code.

The results don't contain specific 2025 information comparing PyTorch standalone binary handling between commercial and open source Nuitka versions. For the most current details on commercial pricing and specific features, you may want to check nuitka.net/doc/commercial.html directly.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

8. assistant (2026-06-21T13:52:32.209Z)

Let me read the relevant docs and continue research on the other tools and key issues.

9. user (2026-06-21T13:52:33.209Z)

```
1      # Cross-Platform Desktop App ? "Plug in your card, get your highlights"
2
3      > **Status:** `DRAFT` ? `owner-gated` (scoping doc only; **nothing is built**) .
**Owner:** avidullu . **Created:** 2026-06-18 . **Last updated:** 2026-06-18
4      > **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5      > **Tracking anchors:** $7 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once the owner
greenlights P0).
6      > **Relation to existing docs:** **peer-of**
[ `PLATFORM_ARCHITECTURE.md` ] (PLATFORM_ARCHITECTURE.md) (this is the **`LocalDesktop`
ComputeBackend** made into a product) . **realizes the L0 tier** of
[ `VIDEO_LOCALITY_MODEL.md` ] (VIDEO_LOCALITY_MODEL.md) . **consumes**
[ `OWNED_MODEL_IMPLEMENTATION_PLAN.md` ] (OWNED_MODEL_IMPLEMENTATION_PLAN.md) (the smaller/export-
clean model is one answer to the compute crux) . bound by
[ `COMMERCIALIZATION.md` ] (COMMERCIALIZATION.md) licensing guardrails.
7      > **Honesty note:** every claim is tagged `[verified]` (checked against code/measured),
`[researched]` (needs sign-off), or `[design]` (proposed). **The entire build is `[design]`** ?
```

this doc scopes; it does not ship. Not legal advice.

8

9 ---

10

11 ## 0. TL;DR

12 Package the mature local pipeline (WASB shuttle detector + improved windowing + ffmpeg stitch, already runnable fully-offline via [``tools/generate_highlights.py --skip-ai-handoff``](`../tools/generate_highlights.py`)) as a **cross-platform desktop app** for **non-technical recreational players**: plug in the camera's USB/SD card, click once, get back **only the highlight reels** ? fully on-device, **no upload**, **no copy of the GBs of 4K originals**. This is the product surface that fits the "local, cheap, efficient" thesis and the L0 ("nothing leaves the machine") tier of ``VIDEO_LOCALITY_MODEL.md``.

13

14 **The make-or-break unknown is compute on a typical enthusiast laptop** ? which often has **no discrete GPU**. WASB runs at **~3.4x real-time** on a laptop RTX 2070 Super ``[verified, measured]``; on CPU it is far slower, and Apple Silicon has **no CUDA** at all. **P0** is a feasibility spike that measures CPU and Apple-MPS throughput for a ~10-min clip before any app is built. **If CPU-only is unacceptable on commodity hardware**, the path needs a smaller/faster owned model (ties ``OWNED_MODEL_IMPLEMENTATION_PLAN``) or a tiered "fast-if-GPU, slow-but-works-if-CPU" design. **Everything past P0 is gated on that result.**

15

16 ## 1. Problem & goal

17

18 **Why now / the user.** Target users are **recreational sports enthusiasts**, not tech-savvy. Their match videos are **huge** (GBs of 4K). The cloud-upload product surface fails them on every axis: slow on a typical Indian uplink (``VIDEO_LOCALITY_MODEL.md`` §0: 10 GB ? ~1 h at 20 Mbps, ~4.5 h at 5 Mbps), bandwidth-costly, privacy-fraught, and it bloats both our storage and their drives. The thing the customer actually wants back is **kilobytes of timestamps** ? a highlight reel (``VIDEO_LOCALITY_MODEL.md`` §1: **"timestamps are the product; the video is dead weight"**).

19

20 **The concrete outcome ("good" looks like this):**

21 1. User plugs the GoPro/phone SD card or camera USB into their laptop.

22 2. The app detects the card, lists the match videos on it.

23 3. User clicks **Generate Highlights**.

24 4. The app makes an on-device proxy, runs WASB detection + windowing + stitches the reel ? **100% locally**.

25 5. The reel(s) land in a user-chosen folder. **The originals are never copied off the card and never bloat any drive.**

26

27 No account, no upload, no cloud dependency in the default path. Gemini stays an **optional** "better brain if you want it" toggle ? never required, never bundled as a hard dependency, never a training source (``COMMERCIALIZATION`` C4).

28

29 **Read to get current:** ``VIDEO_LOCALITY_MODEL.md`` (the L0/locality framing + Q05 packaging seed), ``PLATFORM_ARCHITECTURE.md`` §3.2 (the ComputeBackend seam), [``backend/pipeline/detectors/base.py``](`../backend/pipeline/detectors/base.py`) (the detector ABC that already anticipates a Linux-native runner), ``tools/generate_highlights.py`` (the local entrypoint this app wraps).

30

31 ## 2. Decisions locked

32

33 | # | Decision | Source / date | Implication |

34 |---|-----|-----|-----|

35 | D1 | **Default path is 100% on-device, no upload, no Gemini.** The reel is produced from the local pipeline (``--skip-ai-handoff``). | This doc · 2026-06-18; matches ``VIDEO_LOCALITY_MODEL`` L0 | The app must never **require** a network call. Gemini is an opt-in toggle only. |

36 | D2 | **Never copy the originals.** The app reads source video in place (off the card / a chosen folder) and writes **only** reels + a tiny report to a user folder. | This doc; mirrors ``generate_highlights.py`` non-destructive ``_atomic_publish`` discipline | No "import library" that duplicates 10 GB files. Detection runs on a proxy; stitch reads the original in place. |

37 | D3 | **Drop the WSL2 dependency; run the detector natively per-OS.** | This doc; enabled by the ``DetectorRunner`` ABC ``[verified]`` | Windows-native (no WSL), Linux+NVIDIA (CUDA directly), macOS (MPS or CPU). The WSL adapter stays for the current dev box but is **not** the

```

shipped path. |
38 | D4 | **Commercial-clean licensing for everything bundled: MIT / Apache-2.0 / BSD
only** (code, weights). ffmpeg ships as an **LGPL build** (no GPL-only encoders like x264). |
`COMMERCIALIZATION` (ratified 2026-06-09); `VIDEO_LOCALITY_MODEL` Q05 (**ffmpeg LGPL build =
licensing-clean**) | WASB-SBDT weights are MIT `[verified]` (see `tracknet-wasb-wsl2-decision`).
The stitch must use openh264 (BSD) or OS hardware encoders, **not** statically-linked x264. See
$3.4. |
39 | D5 | **Reuse the existing pipeline verbatim; add only the shell + a native runner +
device-I/O.** | `backend/pipeline/detectors/base.py` reuse contract `[verified]` | No re-
architecture of windowing/stitch. New code = native runner, USB/SD detection, app shell,
packaging. |
40 | D6 | **Feasibility is gated on P0.** No app shell, no packaging work starts until the
compute spike returns a verdict. | This doc; `decision-rigor-norm` (NUMBERS INFORM, FOUNDATIONS
DECIDE) | P0 is the only un-gated deliverable; P1?P3 are owner-greenlit *after* P0 numbers. |
41
42 ## 3. Foundation ? research / prior art `[design]` / `[researched]`
43
44 ### 3.1 The compute reality (the crux) ? what we know vs must measure
45 - **WASB on a discrete laptop GPU is acceptable** `[verified, measured]`: on the owner's
**RTX 2070 Super Max-Q (8 GB)**, WASB inference is the pipeline driver at **~3.2?3.4x real-
time** (?3.4 GPU-min per footage-min, ~0.055 s/frame), peak VRAM ~5.7?6.0 GB; proxy (NVENC)
~0.2x RT; 4K stitch (CPU libx264) ~16 s/rally; end-to-end ?5x clip length. Source: `current-
state.md` `COST_SCALING.md` checkpoint (6 Boxhill clips, 2026-06-18).
46 - **The typical enthusiast laptop has NO discrete GPU.** A 10-min 4K clip at 3.4x RT is
~34 GPU-min *with* a 2070S. On integrated graphics / CPU-only the wall-clock is unknown and
could be 10x+ worse ? possibly **hours per clip**, which would kill the one-click promise.
47 - **Apple Silicon has no CUDA.** WASB/TrackNet weights are PyTorch; PyTorch MPS (Metal)
backend *may* run them, but op-coverage and speed on the WASB model are **unverified**
`[design]`. CPU fallback on Apple is also unverified.
48 - **Owned-model tie-in.** `OWNED_MODEL_IMPLEMENTATION_PLAN.md` already states the on-
device answer: *train in Python ? **export across the boundary** (ONNX / Core ML / ExecuTorch /
GGUF) ? a thin device shell runs the artifact with zero Python*, with the standing constraint to
**train with export-clean ops**. A smaller, exported, quantized shuttle detector is a credible
answer if the CPU/MPS numbers on the current WASB weights are bad. **This makes P0 partly a
referendum on whether the desktop product needs the owned-model track.**
49
50 ### 3.2 The de-WSL native runner is already designed-for `[verified]`
51 The detector layer was deliberately built to make this port low-regret.
[`backend/pipeline/detectors/base.py`](../backend/pipeline/detectors/base.py) documents,
verbatim, an **"Optional Linux-native path (the main de-risk goal)"**: a
`LinuxNativeRunner`/`ONNXRunner` that **"Skip[s] all wsl / _stage / conda activate, load[s]
weights locally (torch or onnx)? write[s] the exact same CSV format so
`trajectory_to_action_windows` works verbatim."** The WSL plumbing lives in a separate
`WslCommandMixin`, explicitly so a native runner **must not inherit WSL plumbing.**" The actual
inference math is already isolated in
[`wasb_infer.py`](../backend/pipeline/detectors/wasb_infer.py) (torch + the WASB-SBDT modules,
emitting `Frame,Visibility,X,Y`). **So the port is: lift `wasb_infer.py` out of WSL, load
weights with native torch/onnxruntime per-OS, keep the identical CSV contract.** The windowing
brain (`trajectory_to_action_windows`) and stitch are untouched.
52
53 ### 3.3 Compute backends to evaluate in P0
54 | Path | OS | Toolkit | Status |
55 |---|---|---|---|
56 | CUDA-native | Windows-native, Linux+NVIDIA | torch+CUDA (no WSL) | proven in WSL
`[verified]`; native = port |
57 | Apple MPS | macOS (Apple Silicon) | torch MPS / Core ML export | **unverified ? P0
must measure** |
58 | CPU | all (the no-GPU laptop) | torch-CPU / **onnxruntime** (often fastest on CPU) |
**unverified ? P0 must measure; the gating number** |
59 | Smaller/exported model | all | ONNX/Core ML/ExecuTorch, quantized | deferred to owned-
model track; P0 informs whether it's *needed* |
60
61 ### 3.4 App-shell prior art & licensing landscape `[researched]`
62 - **App shell options:** **Tauri** (Rust + system webview; ~3?10 MB binaries;
MIT/Apache) vs **Electron** (bundled Chromium; ~100?150 MB; MIT) vs **CLI + installer** (wrap

```

```

`generate_highlights.py`; smallest effort). All shell licenses are clean.
63     - Backend packaging: the Python/FastAPI backend bundles via PyInstaller or a
frozen venv as a local "sidecar." Packaging was already modernized ? PEP 621 pyproject.toml
+ an indexer console entry point shipped (DEFERRED_HARDENING_PLAN` #7, DONE, PR #167)
[verified] ? so a CLI MVP has a real head start.
64     - ffmpeg licensing (a genuine landmine): ffmpeg is LGPL/GPL. The current stitch path
uses libx264 (GPL). Redistributing that inside an installer would impose GPL on the bundle.
Mitigation (D4): ship an LGPL ffmpeg build using openh264 (BSD) or the OS hardware
encoder (NVENC / Apple VideoToolbox / Linux VAAPI) ? the desktop app should prefer
hardware H.264 anyway (faster, no CPU stitch tax).
65     - Weights: WASB-SBDT = MIT [verified] (tracknet-wasb-wsl2-decision),
redistributable.
66
67     ## 4. Design / architecture [design]
68
69     ### 4.1 The one-click flow (and what code each step reuses)
70     ```
71     [card inserted]
72     ? (NEW) per-OS device-insert watcher ? mount path
73     ?
74     [list videos on the card] ? glob video exts; reuse _VIDEO_EXTS from
generate_highlights.py
75     ?
76     [make on-device proxy] ? ffmpeg downscale (hardware encoder); reuse the proxy +
--detect-from split (#205)
77     ?
78     [detect: WASB native] ? (NEW) NativeWasbRunner (DetectorRunner subclass) ? NO WSL;
same CSV contract
79     ?
80     [windowing ? rallies] ? REUSE trajectory_to_action_windows + improved milestone
windowing (R1: cap=6 + R4)
81     ?
82     [stitch reel from ORIGINAL] ? REUSE VideoCompiler
(backend/pipeline/stitcher/compiler.py); read original in place
83     ?
84     [export reel + tiny report to a user folder] ? reuse _atomic_publish non-destructive
write; NEVER copy originals
85     ```
86     The detector runs on a proxy; the stitch reads the original on the card (the
--detect-from` split, already shipped [verified] in generate_highlights.py). Rally
timestamps transfer 1:1 because the proxy shares the timeline (the collector frame-identical
proxy contract, VIDEO_LOCALITY_MODEL` $1).
87
88     ### 4.2 Reuse map ? what's NEW vs REUSED
89     | Concern | Status | Code |
90     |---|---|---|
91     | Trajectory?rally windowing | REUSE [verified] |
tracknet_runner.trajectory_to_action_windows` (model-agnostic staticmethod) |
92     | Stitch / reel compile | REUSE [verified] |
backend/pipeline/stitcher/compiler.py::VideoCompiler` (pure ffmpeg) |
93     | Local orchestration (proxy?detect?stitch, non-destructive, resumable, --skip-ai-
handoff`, --detect-from`) | REUSE [verified] | tools/generate_highlights.py` |
94     | Detector runner contract | REUSE [verified] |
backend/pipeline/detectors/base.py::DetectorRunner` ABC |
95     | WASB inference math | REUSE (lift out of WSL) [verified] |
backend/pipeline/detectors/wasb_infer.py` |
96     | Native (de-WSL) WASB runner, per-OS | NEW [design] | NativeWasbRunner` ?
sibling of wasb_runner.py, no WslCommandMixin` |
97     | USB/SD insert detection + video listing | NEW [design] | per-OS: Win
WMI/WM_DEVICECHANGE`, macOS DiskArbitration, Linux udev/udisks2 |
98     | App shell / one-click UI | NEW [design] | Tauri vs Electron vs CLI ($3.4; P2
decides) |
99     | Packaging + code-signing per OS | NEW [design] | PyInstaller/MSIX (Win),
.dmg+notarize (mac), AppImage/Flatpak (Linux) ? P3 |
100

```



```

101     ### 4.3 Where this sits in the platform architecture
102     This **is** the `local` / `LocalDesktop` **ComputeBackend** of
`PLATFORM_ARCHITECTURE.md` §3.2, realized as a shippable product instead of a server worker. The
control-plane/data-plane split degenerates cleanly to "all on one machine." Nothing here
forecloses the hosted SaaS path ? it's the privacy-max end of the same spectrum, and a genuine
differentiator (no competitor offers true on-device compute except SwingVision, which proves
it's commercially viable ? `PLATFORM_ARCHITECTURE` §intro).
103
104     ### 4.4 App-shell recommendation (to confirm at P2)
105     Lean: **start with a CLI + installer MVP** (wraps the existing
`indexer`/`generate_highlights.py` entry point ? fastest way to validate packaging, signing,
USB-detect, and the native runner end-to-end on all three OSes), then **graduate to a Tauri
GUI** for the non-technical user (tiny download fits the "huge-video laptops, slim app" frame;
the Python backend runs as a sidecar the Tauri front-end calls over localhost). Electron is the
fallback if webview inconsistencies bite. This mirrors the repo's "simplest thing first,
graduate on evidence" bias.
106
107     ## 5. Risk table `[design]`
108     | Risk | Severity | Mitigation / where decided |
109     |---|---|---|
110     | **CPU/MPS too slow on a no-GPU laptop** (kills the one-click promise) | **make-or-
break** | **P0 spike measures it before anything is built** (§6, §7-P0). Fallbacks: tiered
fast/slow UX, or the owned smaller model. |
111     | ffmpeg GPL contamination via libx264 in the installer | high (legal) | D4: LGPL build
+openh264/hardware encoders only. |
112     | Code-signing friction (mac notarization mandatory; Win SmartScreen without EV cert) |
medium | P3 scopes certs/notarization per OS; budget the Apple Developer Program + a Windows
cert. |
113     | Privacy regressions (the on-device promise is the *selling point*) | medium | D1/D2:
no upload, no original-copy, no required network. Make the "nothing leaves your machine"
guarantee testable (assert no outbound calls in default mode). |
114     | Per-OS device-insert flakiness / removable-media edge cases (card pulled mid-run) |
medium | P1/P2 handle gracefully; never write to the card; fail safe. |
115     | Support burden for non-technical users (drivers, "it didn't detect my card") | medium
| favors a polished GUI + clear errors (reuse the actionable-error discipline already in the WSL
adapter). |
116
117     ## 6. Honest limits ? what this does NOT do
118     - **A DRAFT/DONE status here proves nothing about runtime feasibility.** The doc is a
scope; only the P0 numbers decide whether the product is buildable on commodity hardware.
119     - **If CPU-only is too slow on a typical laptop, the simple "wrap the pipeline" path
does not exist** ? it becomes "ship a smaller owned model" (a much larger effort, ties
`OWNED_MODEL_IMPLEMENTATION_PLAN`) or "GPU-tier only" (shrinks the addressable market). The doc
names this honestly rather than assuming the wrap is free.
120     - **No accuracy claim changes.** The desktop app inherits exactly the current rally-
detection quality (R1 milestone: precision is still the open gap per
`RALLY_DETECTION_GAP_CLOSURE.md`); packaging does not improve detection.
121     - **Gemini-quality reels need the optional online toggle.** The fully-local default uses
`--skip-ai-handoff`; users wanting the Gemini brain accept that ?1280px chunks leave the machine
(consent language, `VIDEO_LOCALITY_MODEL` QQ3).
122     - **macOS support is contingent on P0's MPS/CPU verdict** ? it is not assumed.
123
124     ## 7. Deliverables & progress tracker ? **source of truth**
125     Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**,
branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands.
**Everything below is owner-gated; only this doc (the scoping PR) is in flight today.**
126
127     | ID | Deliverable | Depends on | Gated? | Status | PR |
128     |---|---|---|---|---|---|
129     | **D0** | **This scoping doc** (`DESKTOP_APP_PLAN.md` + README index) | ? | No | ? |
*(this PR)* |
130     | **P0** | **Compute-feasibility spike (make-or-break).** Measure WASB throughput for a
~10-min clip on: (a) CPU-only (torch-CPU & onnxruntime), (b) Apple MPS, (c) confirm CUDA-native
(no WSL). Output: a wall-clock table + a go/no-go verdict + "do we need a smaller model?"
recommendation. **No app code.** | D0 | **Owner greenlight** | ? | ? |

```

```

131 | **P1** | **De-WSL native detector port** (`NativeWasbRunner`, per-OS): CUDA-native
(Win/Linux), MPS/CPU fallback, identical `Frame,Visibility,X,Y` CSV; reuse windowing verbatim;
healthcheck per backend. | P0 verdict | Yes | ? | ? |
132 | **P2** | **Desktop app shell + one-click flow**: USB/SD detect ? list ? proxy ? native
detect ? stitch ? export; shell choice (CLI MVP ? Tauri) confirmed; "no upload / no original-
copy" enforced & testable. | P1 | Yes | ? | ? |
133 | **P3** | **Packaging + code-signing installers** per OS (Win MSIX/Authenticode, mac
.dmg + Developer-ID notarization, Linux AppImage/Flatpak); LGPL ffmpeg bundling; MIT/Apache/BSD
audit of the bundle. | P2 | Yes | ? | ? |
134
135 ## 8. Open questions ? owner / external
136 **Blocking gates (owner decides before the dependent phase):**
137 1. **(gates P1?P3) Greenlight P0?** ? the spike is cheap and decides everything.
Recommend: yes, run P0 next.
138 2. **(gates P1) If CPU is too slow, which fork?** tiered GPU-fast/CPU-slow UX · invest
in a smaller exported owned model (`OWNED_MODEL_IMPLEMENTATION_PLAN`) · GPU-tier-only product
(narrower market).
139 3. **(gates P2) App shell:** confirm CLI-MVP-then-Tauri, or go straight to a GUI /
Electron.
140 4. **(gates P3) Distribution & signing budget:** Apple Developer Program (mandatory for
notarization), a Windows code-signing cert (EV for clean SmartScreen) ? owner cost decision.
141
142 **Nice-to-knows:**
143 5. OS coverage priority for v1 (Windows-first? given the owner's box and the academy
beachhead).
144 6. Auto-update strategy (Tauri updater / Sparkle / none for the pilot).
145 7. Does the desktop app reuse the optional Gemini toggle at all, or stay strictly local
for v1?
146
147 ## 9. Definition of done
148 This subproject is DONE (? flip Status, archive) when:
149 - [ ] **P0** produced a measured throughput table (CPU / MPS / CUDA-native) and an
owner-accepted go/no-go verdict (incl. whether a smaller model is required). *(P0 alone is a
valid stopping point if the verdict is "not feasible without the owned model" ? then this doc
hands off to `OWNED_MODEL_IMPLEMENTATION_PLAN`.)
150 - [ ] **P1** the native (de-WSL) detector runs on ?1 GPU OS and ?1 CPU/MPS path,
producing byte-compatible trajectories (regression vs the WSL runner on a shared clip).
151 - [ ] **P2** a non-technical user can insert a card and get a reel with no terminal, no
upload, and no copied original ? verified end-to-end on ?1 OS.
152 - [ ] **P3** signed/notarized installers exist for the greenlit OSes; the bundle passes
an MIT/Apache/BSD (+ LGPL-ffmpeg) license audit.
153 - [ ] $7 rows all ? and the headline feasibility verdict is recorded for posterity.
154
155 ## 10. References
156 **Internal code anchors `[verified]`:**
[`tools/generate_highlights.py`](../tools/generate_highlights.py) (local orchestration; `--skip-
ai-handoff`, `--wasb-stream`, `--detect-from`, `_WSL_DETECTORS`, `_atomic_publish`) ·
[`backend/pipeline/detectors/base.py`](../backend/pipeline/detectors/base.py) (`DetectorRunner`
ABC + the documented Linux-native de-risk path + `WslCommandMixin`) ·
[`backend/pipeline/detectors/wasb_runner.py`](../backend/pipeline/detectors/wasb_runner.py) (the
WSL adapter to replace) ·
[`backend/pipeline/detectors/wasb_infer.py`](../backend/pipeline/detectors/wasb_infer.py) (the
torch inference core to lift native) · [`backend/pipeline/detectors/tracknet_runner.py`](../back
end/pipeline/detectors/tracknet_runner.py) (shared `trajectory_to_action_windows`) ·
[`backend/pipeline/segmenters/wasb_hybrid.py`](../backend/pipeline/segmenters/wasb_hybrid.py) ·
`backend/pipeline/stitcher/compiler.py::VideoCompiler` · `pyproject.toml` + `indexer` console
entry (#167).
157 **Internal docs:** [`VIDEO_LOCALITY_MODEL.md`](VIDEO_LOCALITY_MODEL.md) (L0 fully-local
tier; Q05 packaging) · [`PLATFORM_ARCHITECTURE.md`](PLATFORM_ARCHITECTURE.md) §3.2
(ComputeBackend seam) ·
[`OWNED_MODEL_IMPLEMENTATION_PLAN.md`](OWNED_MODEL_IMPLEMENTATION_PLAN.md) (export-clean on-
device model) · [`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) (C4 paid-LLM no-train;
MIT/Apache/BSD-only) · [`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md) (#7 packaging,
DONE) · [`TRACKNET_WSL_SETUP.md`](TRACKNET_WSL_SETUP.md) (the WSL setup being superseded for
shipped builds).

```

```
158      **Measured source:** `current-state.md` 2026-06-18 `COST_SCALING.md` checkpoint (WASB
~3.4x RT on RTX 2070 Super Max-Q, peak VRAM ~5.7?6.0 GB).
159      **External (to validate at P0/P2/P3):** PyTorch MPS backend · onnxruntime
(CPU/CoreML/DirectML execution providers) · Tauri vs Electron sizing · ffmpeg LGPL build /
openh264 · Apple Developer-ID notarization · Windows Authenticode/MSIX.
160
161      ---
162
163      ### Changelog
164      - `2026-06-18` ? Created. Scoping doc for the cross-platform desktop app + the de-WSL
native-compute port; all `[design]`. $7 P0 (compute-feasibility spike) named the make-or-break
gate. Owner-gated; nothing built.
165
```

10. user (2026-06-21T13:52:34.315Z)

C:\Users\avidu\Projects\khelsutra\docs\LOCAL_APPLIANCE_ARCHITECTURE.md

11. user (2026-06-21T13:52:39.517Z)

Web search results for query: "PyInstaller GPL license commercial closed source application exception"

Links: [{"title": "License ? PyInstaller 6.20.0 documentation", "url": "https://pyinstaller.org/en/stable/license.html"}, {"title": "License ? PyInstaller 4.2 documentation", "url": "https://pyinstaller.org/en/v4.2/license.html"}, {"title": "Pyinstaller license issue", "url": "https://groups.google.com/g/pyinstaller/c/dDXwMMnAtQ8/m/-DdGuiNoBAAJ"}, {"title": "Pyinstaller prunes license text of included libraries and packages · Issue #5666 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/5666"}, {"title": "License ? PyInstaller 3.3.1 documentation", "url": "https://pyinstaller.org/en/v3.3.1/license.html"}, {"title": "LGPL and GPL License Compliance with PyInstaller - velovix", "url": "https://velovix.github.io/post/lgpl-gpl-license-compliance-with-pyinstaller/"}, {"title": "FAQ · pyinstaller/pyinstaller Wiki · GitHub", "url": "https://github.com/pyinstaller/pyinstaller/wiki/FAQ"}, {"title": "pyinstaller/doc/license.rst at develop · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/blob/develop/doc/license.rst"}, {"title": "Licenses, esp GPL readline with onefile · Issue #4471 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/4471"}, {"title": "License ? PyInstaller bundles Python applications", "url": "https://raw.githubusercontent.com/pyinstaller/pyinstaller.github.io/master/license.html"}]

Based on the search results, here's what I found about PyInstaller's GPL license and its exception for commercial closed-source applications:

PyInstaller's License Structure

PyInstaller is distributed under a dual-licensing scheme using both the GPL 2.0 License, with an exception that allows you to use it to build commercial products, and the Apache License, version 2.0, which only applies to a certain few files.

Commercial Closed-Source Exception

PyInstaller is distributed under the GPL license with a special exception which allows to use PyInstaller to build and distribute non-free programs (including commercial ones). Specifically:

- You may use PyInstaller to bundle commercial applications out of your source code. The executable bundles generated by PyInstaller from your source code can be shipped with whatever license you want, as long as it complies with the licenses of your dependencies.
- You don't need to distribute your source code, and you can ship the executables created with PyInstaller with whatever license you want.
- You do not need to include PyInstaller's license file with your application, nor do you have to provide any form of acknowledgements or credits to PyInstaller.

The key limitation is that the executable bundles must comply with the licenses of your dependencies, meaning you cannot use PyInstaller to bundle GPL-licensed dependencies if you want to distribute closed-source applications.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

12. assistant (2026-06-21T13:52:42.723Z)

Let me read the live appliance architecture doc and research the remaining tools (cx_Freeze, Briefcase, antivirus, code-signing, multiprocessing/uvicorn).

13. user (2026-06-21T13:52:43.648Z)

```
1      # Single-box appliance ? UI-powered rally indexer + reel generator (KhelSutra Studio)
2
3      > **Status:** `DRAFT` ? owner-gated (auth choice, retention, cost ceiling, GPU host, P5
bridge) . **Owner:** Avi . **Created:** 2026-06-20 . **Last updated:** 2026-06-20
4      > **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5      > **Tracking anchors:** $9 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6      > **Relation:** peer-of `PER_RALLY_SELECTION.md` / `REQUEST_ACCESS_FORM.md` in this
repo; **consumes** the engine's decoupled `StorageBackend` + `python -m backend.worker` seams
and the FastAPI loop; **extends** the engine's `HOSTING_PLAN.md` gating + `UI_PROPOSAL.md` alpha
screens. Engine-side changes here are **proposals to coordinate** with the active
decoupled/worker sessions ? not written unilaterally.
7      > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine
source this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not
legal/financial advice.
8
9      ---
10
11     ## 0. TL;DR
12     **KhelSutra Studio** turns the engine from a CLI into a **single-box, UI-powered rally
indexer + reel generator**: the khelsutra `web/` React SPA, served behind an **edge auth gate**,
drives the **existing** engine FastAPI loop (ingest ? index ? pick rallies ? reel ? download) on
**one machine**, with `StorageBackend=local` for the MVP. It is the **single-box deployment
profile of the just-shipped decoupled architecture** ? storage=local + compute=local is the
degenerate collapse of the same storage/worker split.
13
14     The **single most important caveat:** the engine has **zero app-layer auth and CORS
`***` `[verified backend/main.py:42]`, so safety is structural **only** because the engine stays
bound to `127.0.0.1:8000` `[verified backend/main.py:638]`, a firewall blocks inbound `8000`,
and a reverse proxy/tunnel is the sole authenticated ingress ? **the gate, not the app, is the
tenancy boundary.**
15
16     Two honest hardware truths: (1) the current droplet is **CPU-only** ? real detection
there is **Gemini (cloud)** or **stub (CPU mock)**; **real local-GPU detection needs a GPU
host** (native WASB, in-flight). (2) The owner priority is to **stand up a GPU host ASAP** ?
provider-agnostic (DO GPU droplet / GCE GPU VM / Cloud Run+GPU) ? but that runs **in parallel**
with the UI MVP, which ships on Gemini today.
17
18     ## 1. Problem & goal
19     The decoupled engine just shipped a `StorageBackend` ABC + a `python -m backend.worker
{infer|train}` dispatcher `[verified backend/storage/, backend/worker/__main__.py]`, and the
FastAPI loop already covers `process` ? jobs ? query ? compile ? highlights` `[verified
backend/main.py]`. What's missing is a **console** that lets a non-CLI operator run the whole
loop on a single box, **honestly, behind a gate**.
20
21     **Headline CUJ (owner's words):** a **hosted website** where I pick an input ? **a
Google Drive file, a bucket file (`s3:///`/`gs:///`), or a local upload** ? it lands in the
**bucket (DO Spaces / GCS)** or locally ? I **generate highlights** on a **GPU-enabled host**
running the real e2e ? I **pick rallies, watch, and download** ? **completely from the UI.**
```

```

22
23     **Outcome ("good"):** an operator signs in, picks a source, watches it index, curates
rallies, downloads a reel ? on the **live CPU droplet today** (Gemini/stub), and on a **GPU host
later** (native WASB) with **no UI fork**.
24
25     **Read to get current:** `PER_RALLY_SELECTION.md` ($3/$4/$10 contract anchors), engine
`HOSTING_PLAN.md` (gating), `DECOUPLED_COMPUTE/HANDOFF.md` (what's built),
`VIDEO_LOCALITY_MODEL.md` (privacy/retention).
26
27     ## 2. Decisions locked
28     | # | Decision | Source / date | Implication |
29     |---|---|---|---|
30     | D1 | Reverse-proxied **single origin**; engine stays `127.0.0.1:8000` | `[verified
main.py:638]`; 2026-06-20 | CORS `` is moot from the browser ? the proxy is the boundary |
31     | D2 | **Edge auth**, zero engine auth code | `HOSTING_PLAN.md`; 2026-06-20 | Caddy
basicauth OR cloudflared + Cloudflare Access |
32     | D3 | MVP drives the **in-process FastAPI worker** loop, `storage=local` | `[verified
job_worker.py:37, local.py:33]`; 2026-06-20 | bucket worker = reserved scale path, out of MVP |
33     | D4 | Embed **RallyPicker** from `PER_RALLY_SELECTION.md` **verbatim** | PR [#11];
2026-06-20 | the console is a superset; do not redesign the picker |
34     | D5 | Console lives in the khelsutra **web/** React scaffold | `web/README.md`;
2026-06-20 | Cloudflare-Pages-style root, not engine static |
35     | D6 | Compute placement is a **setting**, rendered from `GET /api/capabilities` |
`[design]`; 2026-06-20 | UI offers ONLY what the box can honestly do |
36     | D7 | **Ingest is a source menu** (local upload · bucket URI · Drive) normalizing to
one video object | `[verified backend/storage/ref.py parse_uri]`; 2026-06-20 | Drive is a
**stub** today ? scoped honestly ($4.2) |
37     | D8 | GPU host is **provider-agnostic** (DO droplet / GCE VM / Cloud Run+GPU); storage
stays the bucket | Owner 2026-06-20 | compute and storage are separate planes; no provider lock-
in |
38     | D9 | **Observability is first-class** ? structured logs on every new path + a DoD item
| Owner 2026-06-20 | see $5; verified during CUJ replay |
39     | D10 | Delivery = a **self-hosted webserver + UI the user spins up** (NOT a desktop
app); the user points at **local storage + GPU/CPU OR cloud locations** and the UI drives the
tool | Owner 2026-06-20 | **supersedes `DESKTOP_APP_PLAN.md`** ; one artifact (this appliance)
runs on the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings |
40
41     ## 3. Foundation ? what the engine gives us today `[verified this session unless
noted]`
42     - **FastAPI loop:** `POST /api/process` (202 + `job_id`) ? poll `GET /api/jobs/{id}` ?
`POST /api/query` ? `POST /api/compile` ? `GET /api/highlights/{video_id}/{filename}`
(FileResponse) `[main.py]`. **Chunked upload exists** ? `POST /api/upload/init` · `PUT
/api/upload/{id}/part/{i}` · `POST /api/upload/{id}/complete` · `DELETE /api/upload/{id}`
`[verified uploads.py:64,89,123,147]` ? but the bundled `app.js` **never calls them** (it only
hits `/api/process` + `/api/compile` `[verified app.js:88,373]`). So browser upload is **client-
wiring only**, no engine change.
43     - **In-process worker:** one module-global `ThreadPoolExecutor(max_workers=1)`
`[verified job_worker.py:37]` ? strictly **one job at a time** ; the DB `jobs` table is the
clock; `fail_stale_jobs` sweeps queued/running on boot. Gemini chunk concurrency is a
**separate** knob `indexing.ai_max_workers` (**default 5** `[verified models.py:183]`; set to 1
to serialize) ? it is **not** already 1.
44     - **Storage:** `storage.backend` default = "local" `[verified models.py:298]`;
`LocalStorage.fetch_to_local` is **identity** `[verified local.py:33]`, `put` no-ops on
`src==dst` ? so on a single box the worker's pull?pipeline?push **collapses to read-local/write-
local**. URI scheme `parse_uri` `[verified ref.py]`: bare/local:// ? local; `s3://`
(AWS/R2/**DO Spaces**/MinIO, by endpoint) ; `gcs://`/`gs://` ; `gdrive://` ? **follow-up stub
(emulated only)**. Credentials never in config ? boto3 default chain / `~/aws` `[verified
s3.py]`.
45     - **Compute seam:** `indexing.detector_impl` ? `{auto=WSL, native=Linux-native,
`stub`=CPU mock}; default "auto" `[verified models.py:129]`, resolved in
`trajectory_hybrid._resolve_runner` `[verified :76-85]`. **Native WASB is in-flight on branch
`feat/native-wasb-runner` : ** `native_wasb_runner.py` **exists** `[verified]`;
**`fusion_hybrid._build_native_runner` returns `NativeWasbRunner`** `[verified
fusion_hybrid.py:86]` while **`trajectory_hybrid._build_native_runner` still raises** `[verified
:57-63]`. Acceptance gate = **owner byte-parity** (WSL CSV == native CSV) on a real GPU box ?

```

```

not exercisable on the droplet or in the offline suite. Treat native as **not production-
verified**.
46 - **Worker CLI:** `python -m backend.worker {infer|train}` pulls a `--video-uri` ?
pipeline ? pushes `outputs/<video_id>/{intervals.csv,report.json}` `[verified
worker/___main__.py:88,149]`. The validated GPU-free/key-free combo: `detector_impl=stub` +
`--set indexing.skip_ai_handoff=true` + `storage.s3`.
47 - **Gating:** cloudflared Tunnel (outbound-only, no inbound ports) + Cloudflare Access
email-OTP; backend trusts `Cf-Access-Authenticated-User-Email` `[HOSTING_PLAN.md]`. `GET
/api/health` **now EXISTS** (engine M1, [#268]) ? `{status, sport, default_segmenter}`
`[verified main.py:127, corrected 2026-06-21 ? was "does not exist"]`.
48 - **Droplet reality:** `168.144.126.176`, **CPU-only, NO GPU**, region `blr1`; co-hosts
the marketing site (`/srv/khelsutra-site`, node `:8787`) **and** the engine (`~/rally`); ran the
full train+infer e2e against DO Spaces with the **GPU mocked** `[SETUP_NEW_MACHINE.md,
DECOUPLED_COMPUTE/HANDOFF.md]`. Shared testbed: MinIO `9000/9001` + GCS emulator `9023` also run
there.
49 - **Absent (net-new):** `GET /api/capabilities`, `GET /api/videos` (only single-row
`get_video(id)`), `GET /api/reels/{video_id}`, any source/proxy streaming endpoint, any
**Dockerfile**, any per-video **cost ledger** `[verified ? grep returns none]`.
50
51 ## 4. Design / architecture
52
53 ### 4.1 Topology & request path
54 ONE box: **edge gate** (cloudflared+Access or Caddy basicauth) ? **reverse proxy /
static host** serving the `web/` SPA bundle and proxying `/api/*` ? engine **FastAPI
127.0.0.1:8000** ? **LocalStorage** ? **detector seam** (Gemini/stub today; native WASB on a
GPU host). The firewall blocks inbound `8000` + the test ports `9000/9001/9023`. The marketing
site stays a **separate** systemd unit on a **separate** subdomain.
55
56 ...
57 PUBLIC INTERNET
58 ? (1) EDGE GATE ? the ONLY ingress; auth lives HERE, not in the app
59 ? cloudflared Tunnel + Cloudflare Access (no inbound ports) OR Caddy :443 +
basicauth
60 ?
61 ?????????????????????????????????????????????????????????????????????????????????
62 ? ONE BOX firewall BLOCKS inbound 8000 + 9000/9001/9023 ?
63 ? (2) reverse proxy / static host (Caddy/cloudflared) ?
64 ? /, /assets/* ? web/ React SPA /api/*, /api/highlights/* ? :8000 ?
65 ? ? ONE ORIGIN to the browser ? engine CORS '*' is irrelevant ?
66 ? ? ?
67 ? (3) ENGINE FastAPI (host=127.0.0.1:8000, HARD-bound [main.py:638]) ?
68 ? control: jobs table + ThreadPoolExecutor(max_workers=1) drain ?
69 ? data: uploads.py chunked upload · /api/compile · /api/highlights ?
70 ? ? ?
71 ? (4) STORAGE = LocalStorage (default) fetch=identity · put=no-op(src==dst) ?
72 ? ? ?
73 ? (5) COMPUTE seam: gemini (cloud, key) · stub (CPU mock) [droplet] ?
74 ? native WASB (GPU host) · auto (owner Win+WSL) ?
75 ?????????????????????????????????????????????????????????????????????????????????
76 RESERVED SCALE PATH (out of MVP): console "compute target" selector ? storage=s3 (DO
Spaces)
77 ? SEPARATE GPU host runs `python -m backend.worker infer --video-uri s3:///` ?
outputs/<vid>/ back
78 (needs a NET-NEW orchestration endpoint reflecting outputs into the jobs/poll
contract)
79 ...
80
81 ### 4.2 Ingest sources (the headline) ? one menu, one normalized video object
82 All sources normalize to **"a video the pipeline can read"** ? a local path (MVP) or a
`StorageRef` (scale):
83
84 | Source | Path today | Status |
85 |---|---|---|
86 | **Local upload** | SPA chunks the file ? `max_part_mb` via the existing
`/api/upload/init|part|complete` `[verified uploads.py]` ? server path ? `POST /api/process`.

```

```

(Scale: also `StorageBackend.put` to `s3:///`/`gcs:///`) | **wire client only** (endpoints exist)
|
87 | **Bucket URI** (`s3:///`, `gs:///`) | **POST /api/process` accepts the bucket URI
directly** (resolve ? fetch to scratch ? same pipeline ? API DB) `[verified main.py:159,274,283
? corrected 2026-06-21; was "net-new"]`; the worker CLI also fetches `--video-uri`. So the
**engine endpoint is NOT net-new** ? only the SPA call that POSTs the URI + polls `/api/jobs`
is. *(Default single-user mode; HOSTED mode w/ `allowed_video_root` correctly rejects a bucket
URI 400.)* | **engine: DONE (verified); SPA ingest call: design (= B-P2)** |
88 | **Google Drive** (`gdrive:///`) | `parse_uri` recognizes it, but the gdrive backend is
an **emulated stub** `[verified ref.py, DECOUPLED_COMPUTE research]` | **stub ? roadmap;** MVP
requires a Drive?bucket pre-step; do **not** present Drive as live until the backend is real |
89
90     ### 4.3 Compute placement matrix (one UI, surfaced from `/api/capabilities`)
91     | Placement | Runs | Needs | Real/mock | Honest caveat |
92     |---|---|---|---|---|
93     | **CPU droplet ? gemini** (MVP target) | `segmenter=gemini`, in-proc worker |
`GEMINI_API_KEY`; network; **no GPU** | **REAL** | Cloud brain: uploads ?1280px chunks to Google
? **must be in consent copy**; **no per-video cost ceiling** today |
94     | **CPU droplet ? stub** | trajectory hybrid + `detector_impl=stub`+`skip_ai_handoff` |
nothing | **MOCK** | not real tracking ? UI must label "stub (CPU mock ? not real detection)";
demo/regression only |
95     | **CPU droplet ? motion** | `segmenter=motion` | nothing | REAL, low-quality | only no-
key/no-GPU real segmenter; never surface `server/receiver/events` (fabricated) |
96     | **GPU host ? native WASB** (in-flight) | `wasb`+`detector_impl=native` ?
`NativeWasbRunner` (in-proc torch) | a **GPU host ? DO GPU droplet / GCE GPU VM / Cloud
Run+GPU**`; `WASB_WEIGHTS` | **REAL**, on-device | partially wired (fusion builds it, trajectory
refuses); **owner byte-parity gate**; **Cloud Run/GCE need a containerized worker ? no
Dockerfile exists, and WASB's torch-1.11 conda env makes the image non-trivial** |
97     | **Owner Win+WSL ? auto** | `wasb`+`detector_impl=auto` (WSL2+conda) | Windows+WSL2+GPU
| REAL (today) | Windows-only transport; the working real-GPU path for owner verification |
98     | **Remote GPU worker via bucket** (reserved) | console CPU + `storage=s3`; separate GPU
host runs `backend.worker infer` | bucket creds; a GPU host; a **net-new orchestration
endpoint** | REAL (remote) | FastAPI loop and worker CLI are **separate** today; "remote GPU as
a setting" is **design** ? selector shown **disabled** in MVP |
99
100     **Provider note:** **Cloud Run + GPU** (scale-to-zero, pay-per-job) is the strongest fit
for the engine's own **per-video-billing** economics (`DECOUPLED_COMPUTE` #246) ? gated on the
**containerization feasibility check** above.
101
102     ### 4.4 Auth / exposure (non-negotiable invariant)
103     No engine auth, by design; the gate is external. **Invariant (all three must hold):**
engine bound to `127.0.0.1` + firewall closes `8000` + edge gate up. Go-live checklist
`[HOSTING_PLAN.md:102-108]`: Access/basicauth ON; **rotate the chat-shared `GEMINI_API_KEY`**;
set `security.allowed_video_root`; rate-limit; per-user quota.
104
105     ### 4.5 UI operator console (web/ SPA ? superset of RallyPicker)
106     - **S1 Ingest & Process** ? the source menu ($4.2) + a segmenter picker from
`/api/capabilities` ? `POST /api/process`.
107     - **S2 Progress** ? 3s poll of the free-text stage
(`hashing/validating/segmenting/finalizing`), a **"one job at a time"** notice, and the restart-
failure terminal state.
108     - **S3 Rally pick ? reel** ? **embed RallyPicker verbatim** (`PER_RALLY_SELECTION.md`):
honor `min_shot_count`; never render motion `server/receiver/events` or a fake confidence meter.
**Per-rally is ONE screen of the console.**
109     - **S4 Reels / History** ? needs `GET /api/videos` + `GET /api/reels/{video_id}` for
**reload-survival**.
110     - **S5 Settings** ? render `config` + `capabilities`; the compute/storage **target
selector shown DISABLED** ("single-box: local") until the bucket-worker bridge lands.
111
112     ### 4.6 Reuse map
113     **Reused (no re-plumb):** the full `/api` loop; the in-process worker; the chunked
upload endpoints (un-called); LocalStorage default; the detector seam incl. the landing
`NativeWasbRunner`; the **RallyPicker** design + anchors; the `web/` scaffold; the
`HOSTING_PLAN` gate; the `SETUP_NEW_MACHINE` runbook. **Net-new:** Caddyfile + systemd units;
`GET /api/capabilities`; `GET /api/videos`; `GET /api/reels/{video_id}`; `GET /api/health`; the

```

SPA console screens; ingest-from-bucket wiring; a retention sweep; (reserved) the bucket-worker bridge.

114

115 ## 5. Observability ? logging & monitoring `[required, first-class]`

116 Launch-readiness directive (owner 2026-06-20): ****logs and monitoring taken seriously, with a full log-analysis pass across every new code path.**** The engine already uses a centralized Python logger `[verified main.py:12-17]` ? carry that discipline into every new path.

117

118 - ****Correlation ID end-to-end.**** Thread the existing `job\_id` (from `/api/process`) ****plus**** an `upload\_id`/`request\_id` through ingest ? process ? jobs ? compile ? highlights, and into the worker for the bucket path, so ****one CUJ is traceable**** across UI ? FastAPI ? worker ? GPU host. Log it on every line.

119 - ****Structured logs.**** New endpoints/screens emit structured (JSON-able) logs at ****entry / result / error**** ? not just "it ran". The SPA logs client-side errors + failed fetches with the same correlation id.

120 - ****Load failures, never silent drops.**** Carry the engine's ethos (#240 "surface failed chunks loudly") + RallyPicker's `min\_shot\_count`-warning rule: a partial/empty reel must ****never**** look complete; every dropped clip/chunk is logged + surfaced in the UI.

121 - ****Health + metrics.**** Ship the missing `GET /api/health` `[HOSTING\_PLAN.md:42]` and a thin metrics surface (jobs queued/running/failed, last error, disk free, per-video cost once a ledger exists) for the engine-mode banner + alerting.

122 - ****Aggregation for transient hosts.**** A scale-to-zero Cloud Run / GPU host is ****ephemeral**** ? logs must ship to a durable sink (the bucket or a log service), not die with the instance.

123 - ****Log-analysis pass before merge (DoD item).**** Every PR that adds a code path includes a check that the path logs meaningfully, and the ****logs are inspected during CUJ live-replay**** (\$8 Layers 1?3). Ties [[feedback-launch-quality-bar]].

124

125 ## 6. Risk table

126 | Risk | Today | Mitigation |

127 |---|---|---|

128 | No app auth + CORS `` `[main.py:42]` | open if any of {localhost-bind, firewall, gate} slips | enforce all three; ****LIVE posture test**** that raw `:8000` is refused from outside |

129 | Test ports on shared droplet | MinIO `9000/9001` + GCS emu `9023` | firewall closes them |

130 | Retention | `output`/`+`uploads/` originals ****never deleted**** | sweep / keep-proxy-only before any non-owner exposure |

131 | Surprise Gemini/GPU bill | ****no cost ceiling/ledger**** | per-video cap; `ai\_max\_workers` (default 5) tunable; `max\_workers=1` job serialization |

132 | Native WASB unverified | partially wired; owner-parity-gated | treat as not-production-verified until owner byte-parity on a real GPU box |

133 | New engine endpoints | `capabilities`/`videos`/`reels`/`health` don't exist | coordinate engine PRs with active sessions; ****branch from `origin/master`, re-verify HEAD, explicit `git add`**** |

134 | Drive ingest overstated | `gdrive://` is a stub | don't present Drive as live; require Drive?bucket pre-step until real |

135 | No Dockerfile | runbook-only reproducibility | feasibility-check a worker image before promising Cloud Run/GCE |

136 | Shared-checkout collisions | active worker sessions share the engine tree | the branch-safety protocol on every engine-side PR |

137 | ****Provisioning-credential leak**** | a DO API token / cloud creds could leak if pasted in chat or committed (cf. the ****already-rotated Spaces key**** exposed in chat, `DECOUPLED\_COMPUTE/HANDOFF.md`) | authenticate the CLI ****on-box**** (`doctl auth init` interactive / `gcloud` already authed) so the secret never enters chat or the repo; least-privilege token, revoke after; creds 0600, never committed; ****creating paid GPU infra is a confirm-first action every time**** |

138

139 ## 7. Honest limits ? what this does NOT do

140 Does ****not**** add multi-tenant identity/RLS (single-tenant alpha; uploads/outputs global). Does ****not**** run real WASB on the CPU droplet (Gemini or stub only). Does ****not**** make the remote-GPU bucket path work (design; selector disabled). Does ****not**** provide in-UI source playback, job cancel, or a true progress bar (coarse free-text stage, poll-only). Does ****not**** containerize (runbook; no Dockerfile `[verified]`). Does ****not**** verify native WASB (owner byte-


```

parity on a real GPU box, not this offline suite). A green SPA + contract tests prove the
**console plumbing**, not engine rally-detection quality (a separate track).
141
142     ## 8. CUJs & live-test plan `[required]`
143     **CUJs:** **CUJ-1** ingest (Drive/bucket/local) ? index ? reel ? download (core);
**CUJ-2** re-open a past match after reload; **CUJ-3** pick-by-query (deterministic filters +
honesty-hiding); **CUJ-4** settings/compute placement (truthful capabilities).
144
145     **Live-test plan ? 5 layers, every CUJ replayed LIVE throughout development:**
146     - **L1 ? Mock engine (hermetic, fast, default + CI).** A tiny FastAPI/MSW implementing
the **exact** contract (upload; `process` 202; `jobs` state machine incl. restart-failure;
`query` with a motion fixture for honesty-hiding; `config`; `capabilities` per box profile ?
CPU-no-key / CPU-with-key / GPU; `compile`; `highlights` tiny mp4; the new `videos`/`reels`).
The SPA drives **all 4 CUJs in <1s**, **replayed via the Claude Preview/browser tools** (start
dev server ? fill the ingest picker ? click through process/poll/RallyPicker ? assert the reel
link via snapshot/network). Catches reload-survival (CUJ-2), the `play_segments` field-name bug
(CUJ-3), capability honesty (CUJ-4).
147     - **L2 ? Engine-contract checks (real FastAPI, GPU-free).** Boot `python -m backend.main
--server` with `storage=local` + `detector_impl=stub` + `skip_ai_handoff` and assert the **mock
and the real engine agree shape-for-shape** (202 keys, jobs enum, `result.play_segments` schema,
compile `{status,filepath}`, highlights content-type). Every new endpoint is born with a mock
handler **and** a real-engine contract test **in the same PR**. Runs in CI.
148     - **L3 ? Real local e2e (the MVP substrate).** Through the gate on the droplet: CUJ-1
with `stub` (key-free) and `gemini` (real, short clip to bound cost); the **exposure-safety
posture test** (raw `:8000`/`:900x` refused from outside) as an assertion, not an assumption;
reuse the `parity.yml` dual-host pattern.
149     - **L4 ? CI regression (green gates merge).** `npm test` + a stub-engine contract job +
headless browser-replay CUJ scripts; one small PR per tracker row off `origin/master`, **no
stacking**; coverage at/above the floor; **logs inspected during replay** ($5).
150     - **L5 ? Owner hardware e2e (handed off).** Native WASB **byte-parity** on a real GPU
box; real-4K **golden-corpus** runs (Gemini + native); Modern-Standby/`cv2`-vs-`ffmpeg`
behavior; a multi-GB browser upload. Reported back into $9, stated plainly as **not verified**
in the offline suite.
151
152     ## 9. Deliverables & progress tracker ? **source of truth**
153     Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**,
branched from `origin/master`, no stacking; CI green gates merge; coverage ? floor; each PR
ships its tests + meaningful logs ($5).
154
155     | ID | Deliverable | Depends on | Gated? | Status | PR |
156     |----|-----|-----|-----|-----|----|
157     | T0 | This design doc + `docs/README.md` + `NEXT_STEPS.md` pointers | ? | owner | ? |
(this PR) |
158     | T1 | Mock engine + capability fixtures + browser-replay CUJ scripts (L1) | T0 | no | ? |
| ? |
159     | T2 | Appliance shell: Caddyfile + systemd units; firewall; **edge auth**; rotate key
(P1) | ? | owner | ? | ? |
160     | T3 | L3 live posture test (raw `:8000`/`:900x` refused from outside) | T2 | no | ? | ? |
|
161     | T4 | S1 ingest (local upload wired) + segmenter picker | T1 | no | ? | ? |
162     | T5 | S2 progress (poll + one-job + restart-failure) | T4 | no | ? | ? |
163     | T6 | S3 embed **RallyPicker** + compile + download | T4 | no | ? | ? |
164     | T7 | *(engine)* `GET /api/capabilities` + `GET /api/health` + banner | T1 | coord | ? |
| ? |
165     | T8 | *(engine)* `GET /api/videos` + `GET /api/reels/{video_id}` (S4 reload-survival) |
? | coord | ? | ? |
166     | T9 | S5 Settings (config+capabilities; disabled compute/storage target) | T7,T8 | no |
? | ? |
167     | T10 | Observability pass: correlation id end-to-end + structured logs + metrics ($5) |
T4 | no | ? | ? |
168     | T11 | Retention sweep for `output`/`+`uploads/` | ? | owner | ? | ? |
169     | T12 | Ingest from **bucket URI** (`s3://`/`gs://`) wiring ? **engine half DONE**
(`/api/process` accepts URIs, M2/[#280], `[verified]`); remaining = the SPA ingest call (=
**B-P2**) | T4 | coord | ? | engine done; SPA pending |
170     | T13 | **GPU host stand-up** (provider-agnostic) + wire `detector_impl=native`; owner

```

```

byte-parity (P4) | branch | ? owner | ? | ? |
171 | T14 | *(reserved)* Bucket-worker bridge ? remote-GPU "compute as a setting" (P5) |
T8,T12 | ? owner | ? | ? |
172
173 **Owner priority (2026-06-20): T13 (GPU host) is elevated to run in parallel from the
start** ? owner provisions the host + token; the engine session lands native WASB; the UI MVP
does **not** block on it (ships on Gemini).
174
175 ## 10. Open questions ? owner / external
176 **Blocking gates:** (1) **edge-auth** ? Caddy basicauth (self-contained) vs
cloudflared+Cloudflare Access (matches the two-host design, no inbound ports)? (2) **retention
defaults** ? auto-delete originals after compile? keep proxies+intervals only? window?
(privacy/consent). (3) **per-video Gemini/GPU cost ceiling** ? what cap, hard-fail vs warn? (4)
**GPU host choice** ? DO GPU droplet / GCE GPU VM / Cloud Run+GPU (and is a containerized worker
worth building for Cloud Run)? (5) **P5 bridge ownership** ? who builds the orchestration
endpoint + how it coordinates with the active worker sessions.
177 **Nice-to-knows:** console home (web/ vs engine static); a review-playback streaming
endpoint; per-user scoping timing; containerization vs runbook.
178
179 ## 11. Definition of done
180 - [ ] CUJ-1 works end to end on the droplet (Gemini **and** stub) behind the gate, via
the ingest menu.
181 - [ ] Raw `:8000`/`:9000` unreachable from outside (**tested**, not assumed).
182 - [ ] Local upload wired; bucket-URI ingest at least designed/tracked; Drive scoped
honestly (not faked).
183 - [ ] RallyPicker embedded with honesty rules; `min_shot_count` surfaced.
184 - [ ] `/api/capabilities` + `/api/health` + banner truthful; reload-survival via
`/api/videos` + `/api/reels`.
185 - [ ] **Observability:** correlation id threaded end-to-end; structured logs at every
new path; loud failures; logs inspected during CUJ replay.
186 - [ ] Retention sweep in place; `GEMINI_API_KEY` rotated; `allowed_video_root` set.
187 - [ ] CI green (L2 contract + L4 regression); coverage ? floor; $9 tracker all
??-resolved.
188
189 ## 12. References
190 **Engine code `[verified this session]`:**
`backend/main.py:12-17,42,52,307-328,331-340,342-396,638`;
`backend/api/uploads.py:64,89,123,147`; `backend/api/job_worker.py:37`;
`backend/api/static/app.js:88,344,373`;
`backend/storage/{base.py,local.py:33,ref.py,__init__.py:43}`;
`backend/config/models.py:129,183,298`;
`backend/pipeline/segmenters/{trajectory_hybrid.py:57-63,76-85, fusion_hybrid.py:76-86,
gemini.py:470-485, motion.py:191-198}`; `backend/pipeline/detectors/{native_wasb_runner.py,
stub_runner.py}`; `backend/worker/__main__.py:88,98,149`; `config.json` (`default_segmenter:
gemini`). **Engine docs:** `HOSTING_PLAN.md:42,102-108`; `VIDEO_LOCALITY_MODEL.md`;
`DESKTOP_APP_PLAN.md`; `SETUP_NEW_MACHINE.md`; `PLATFORM_ARCHITECTURE.md`;
`DECOUPLED_COMPUTE/{README,02,04,07,HANDOFF}.md`; `UI_PROPOSAL.md:41-66`;
`PROJECT_DOC_TEMPLATE.md`. **This repo:** `PER_RALLY_SELECTION.md`, `REQUEST_ACCESS_FORM.md`,
`PORTABILITY.md`, `web/README.md`.
191
192 ---
193 ### Changelog
194 - 2026-06-20 ? Initial `DRAFT`. Consolidated three single-box-appliance designs
(FastAPI-native / unified-origin / decoupled-profile) into the reverse-proxied unified-origin
spine + compute-as-a-setting framing, via a multi-agent design pass; every engine-fact anchor
re-verified against source before writing (catching the native-WASB partial-wiring +
`ai_max_workers` default nuances). Folded in the owner's ingest-menu (Drive/bucket/local),
multi-provider GPU, and first-class Observability directives.
195

```

14. user (2026-06-21T13:52:53.807Z)

Web search results for query: "Briefcase Beeware vs cx_Freeze packaging PyTorch large ML app feasibility"

Links: [{"title": "How do I ship a Python application to end users? | pydevtools", "url": "https://pydevtools.com/handbook/explanation/how-do-i-ship-a-python-application-to-end-users/"}, {"title": "Briefcase | pydevtools", "url": "https://pydevtools.com/handbook/reference/briefcase/"}, {"title": "BeeWare Briefcase Packaging: Native Python iOS Android Builds 2026", "url": "https://johal.in/beeware-briefcase-packaging-native-python-ios-android-builds-2026/"}, {"title": "What apps can be built using Python? - Quora", "url": "https://www.quora.com/What-apps-can-be-built-using-Python"}, {"title": "Deploy Pytorch model under Android with Python via BeeWare - PyTorch Forums", "url": "https://discuss.pytorch.org/t/deploy-pytorch-model-under-android-with-python-via-beeware/62201"}, {"title": "Reducing the Size of PyTorch-Based Executable · Issue #2412 · marcelotduarte/cx_Freeze", "url": "https://github.com/marcelotduarte/cx_Freeze/issues/2412"}, {"title": "Creating Directly Executable Cross-Platform GUI Apps with Python Tkinter ? Be on the Right Side of Change", "url": "https://blog.finxter.com/creating-directly-executable-cross-platform-gui-apps-with-python-tkinter/"}, {"title": "cx_freeze problem with pytorch · Issue #2495 · marcelotduarte/cx_Freeze", "url": "https://github.com/marcelotduarte/cx_Freeze/issues/2495"}, {"title": "How to compile pytorch in an executable? - deployment - PyTorch Forums", "url": "https://discuss.pytorch.org/t/how-to-compile-pytorch-in-an-executable/89192"}]

Based on the search results, here's what I found about Briefcase/BeeWare versus cx_Freeze for packaging large ML applications with PyTorch:

Key Findings

****cx_Freeze for Desktop/PyTorch Apps:****

cx_Freeze has the strongest installer story, building MSI files on Windows, DMG files on macOS, and AppImage and Debian packages on Linux through a unified interface. However, a PyTorch install is typically hundreds of megabytes on its own, and full desktop bundles that include it can push past 1 GB.

There are documented challenges with cx_Freeze and PyTorch: users report issues with DLL dependencies and compatibility errors when attempting to freeze PyTorch applications.

****Briefcase/BeeWare for Multi-Platform:****

Briefcase packages Python apps for iOS, Android, macOS, Windows, Linux, and Web from a single pyproject.toml config, producing platform-specific artifacts from a single configuration. However, Briefcase cannot package many C-extension and ML dependencies on mobile targets.

Briefcase faces more established competition on desktop-only targets from PyInstaller, Nuitka, and cx_Freeze.

****Broader Recommendation:****

For large ML applications, the recommended approach for ML models is to export to ONNX and ship onnxruntime, only falling back to bundling PyTorch or TensorFlow if ONNX doesn't cover your operators.

****Bottom line:**** For large PyTorch desktop apps, cx_Freeze is more mature for desktop distribution, but both tools face significant size challenges with PyTorch itself. Consider ONNX export as an alternative to reduce bundle size.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

15. user (2026-06-21T13:52:54.017Z)

Web search results for query: "PyInstaller onefile antivirus false positive Windows Defender code signing 2025"

Links: [{"title": "My --onefile exe is getting anti-Virus False positive flags and is disabled by windows. · Issue #6754 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/6754"}, {"title": "How to Fix Antivirus False Positives with PyInstaller Executables", "url": "https://www.pythonguis.com/faq/problems-with-antivirus-software-and-pyinstaller/"}, {"title": "Pyinstaller - false positive? - Python Help - Discussions on Python.org", "url": "https://discuss.python.org/t/pyinstaller-false-"}]

```
positive/43171"}, {"title": "anti-virus says exe is a threat · pyinstaller · Discussion #5877", "url": "https://github.com/orgs/pyinstaller/discussions/5877"}, {"title": "Compacting using pyinstaller causing antivirus false positives · Issue #711 · upx/upx", "url": "https://github.com/upx/upx/issues/711"}, {"title": "Pyinstaller EXE detected as Virus? (Solutions and Alternatives) - CodersLegacy", "url": "https://coderslegacy.com/pyinstaller-exe-detected-as-virus-solutions/"}, {"title": "The antivirus software reports a virus when packaging with PyInstaller 6.3.0. · pyinstaller · Discussion #8207", "url": "https://github.com/orgs/pyinstaller/discussions/8207"}]]
```

Based on the search results, here's what I found about PyInstaller, onefile, antivirus false positives, and Windows Defender code signing:

The Issue

PyInstaller's --onefile mode creates a single executable that extracts everything to a temporary folder at runtime. This extraction behavior is one of the things that triggers AV heuristics. This extraction-and-execution pattern is exactly what a lot of actual malware does. Malicious software often uses "packers" to compress and hide their payloads, extracting them at runtime to avoid detection. Because PyInstaller's behavior looks similar from the outside, antivirus heuristics can flag PyInstaller-built executables as suspicious.

Code Signing as a Solution

Code signing doesn't make false positives impossible, but it dramatically reduces them, especially with Windows Defender and SmartScreen. Code signing helps here because AV vendors can whitelist your certificate rather than individual file hashes.

Recent 2025 Issues

The search results show that PyInstaller 6.3.0 has more Windows Defender hits on VirusTotal (6/69) compared to version 5.13.2 (2/69). This indicates that Windows Defender false positives remain an ongoing issue with newer PyInstaller versions.

Alternative Solutions

Using --onedir mode produces a folder containing your executable alongside all its dependencies. Since nothing needs to be extracted at runtime, the behavior looks less suspicious to AV software.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

16. assistant (2026-06-21T13:52:58.016Z)

I have strong grounding now. Let me do a few more targeted searches on the specific hard problems: dual-Python-env / conda bundling, uvicorn multiprocessing in frozen apps, and Nuitka commercial pricing/PyTorch maturity.

17. user (2026-06-21T13:53:06.273Z)

Web search results for query: "Nuitka standalone PyTorch onnxruntime success report build time size 2025"

Links: [{"title": "Releases 1.0 ? 1.9 ? Nuitka the Python Compiler", "url": "https://nuitka.net/changelog/Changelog-1.x.html"}, {"title": "Nuitka | pydevtools", "url": "https://pydevtools.com/handbook/reference/nuitka/"}, {"title": "Nuitka Release 1.6 ? Nuitka the Python Compiler", "url": "https://nuitka.net/changelog/posts/nuitka-release-16.html"}, {"title": "ONNX Runtime | Pytorch", "url": "https://onnxruntime.ai/pytorch"}, {"title": "Very long compile times for pytorch standalone build · Issue #754 · Nuitka/Nuitka", "url": "https://github.com/Nuitka/Nuitka/issues/754"}, {"title": "Nuitka Changelog", "url": "https://nuitka.net/doc/Changelog.pdf"}, {"title": "Create standalone nuitka"}]

deployment with only some parts of PyTorch · Issue #2218 ·

```
Nuitka/Nuitka", "url": "https://github.com/Nuitka/Nuitka/issues/2218"}, {"title": "ONNX Runtime |  
onnxruntime", "url": "https://onnxruntime.ai/docs/"}, {"title": "Nuitka Release 1.6 - nuitka  
??", "url": "https://nuitka.0.org.cn/main/update-log/1-6"}]
```

Based on the search results, I found some relevant information about Nuitka's standalone builds with PyTorch and onnxruntime, though the results don't contain a specific "success report" with detailed build time and size metrics from 2025.

Here's what I found:

****Nuitka Support for PyTorch and ONNX Runtime:****

Nuitka has built-in configs for packages that most often break bundlers, including PyTorch. Recent releases added built-in package-specific rules for PyTorch, TensorFlow, scikit-learn, and SciPy that prune unused operator kernels and skip compiling generated code.

****ONNX Runtime Fixes:****

The changelog shows several fixes related to onnxruntime support: onnxruntime had too few DLLs included in standalone builds, and a missing DLL for onnxruntime on Linux was added.

****Build Time and Size Considerations:****

A medium-size application that PyInstaller packages in 30 seconds can take 5 to 15 minutes in Nuitka because every module passes through a C compiler, and output binaries are also larger than PyInstaller equivalents.

However, the search results don't contain a specific 2025 success report with detailed build time and binary size metrics for PyTorch and onnxruntime standalone builds. You may want to check the Nuitka GitHub repository or official documentation for recent benchmarking reports.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

18. user (2026-06-21T13:53:06.550Z)

Web search results for query: "PyInstaller bundle two python environments separate conda interpreter subprocess sidecar"

```
Links: [{"title": "Using PyInstaller to Easily Distribute Python Applications ? Real  
Python", "url": "https://realpython.com/pyinstaller-python/"}, {"title": "How to open a sub process  
with PyInstaller python exe virtual environment? · pyinstaller · Discussion  
#6046", "url": "https://github.com/orgs/pyinstaller/discussions/6046"}, {"title": "Understanding  
PyInstaller Hooks ? PyInstaller 6.21.0  
documentation", "url": "https://pyinstaller.org/en/stable/hooks.html"}, {"title": "How to use  
PyInstaller - Earthly Blog", "url": "https://earthly.dev/blog/pyinstaller/"}]
```

I found search results related to your query. Here's what I found:

Key Findings

****Using PyInstaller with Multiple Executables:****

If you freeze script.py into scriptBundled.exe, you can execute it directly via subprocess.Popen. In that case, it will contain its own copy of the required modules, packages, and other resources.

****Resource Sharing Between Multiple Executables:****

If script.py and main.py share virtual environment, you can also re-utilize shared libraries and data files by building via .spec file that builds two executables in a single dist directory (i.e., use single COLLECT()). That applies to onedir builds; for onefile ones, resource sharing

could be achieved by using `MERGE()` in the `.spec` file.

****PyInstaller's Embedded Interpreter:****

PyInstaller uses embedded python interpreter to run the frozen entry-point script and collected python modules. So there's no external `python.exe` interpreter involved, and there's no externally accessible virtual environment left, either.

****Conda-Specific Support:****

The `conda_support` module is available only if run inside a Conda environment.

The search results suggest that bundling two separate Python environments with PyInstaller typically involves creating separate executables for each environment and launching them as subprocesses, rather than bundling two distinct conda interpreters in a single application package.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

19. user (2026-06-21T13:53:07.143Z)

Web search results for query: "PyInstaller multiprocessing freeze_support uvicorn workers reload frozen executable issues"

Links: [{"title": "Can QtCore.QProcess be freeze_support like multiprocessing · Issue #7512 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/7512"}, {"title": "Common Issues and Pitfalls ? PyInstaller 6.20.0 documentation", "url": "https://pyinstaller.org/en/latest/common-issues-and-pitfalls.html"}, {"title": "Multiprocessing freeze support error on Linux · Issue #7920 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/7920"}, {"title": "Common Issues and Pitfalls ? PyInstaller 6.21.0 documentation", "url": "https://pyinstaller.org/en/stable/common-issues-and-pitfalls.html"}, {"title": "Multiprocessing Freeze Support in Python ? SuperFastPython", "url": "https://superfastpython.com/multiprocessing-freeze-support-in-python/"}, {"title": "GitHub - mohammadhasananisi/compile_fastapi: compile fastapi with pyinstaller · GitHub", "url": "https://github.com/mohammadhasananisi/compile_fastapi"}, {"title": "PyInstaller not working with Multiprocessing · pyinstaller · Discussion #7798", "url": "https://github.com/orgs/pyinstaller/discussions/7798"}, {"title": "uvicorn and fastAPI with pyinstaller failing to start some of the uvicorn workers · Kludex/uvicorn · Discussion #1820", "url": "https://github.com/Kludex/uvicorn/discussions/1820"}]

Here are the search results for PyInstaller multiprocessing freeze_support with uvicorn workers, reload, and frozen executable issues:

Key Findings:

****General Multiprocessing with PyInstaller:****

When your main script uses multiprocessing functionality, the corresponding worker sub-process also needs to execute the remainder of your script, including the imports, which limits the performance benefits of an early `freeze_support()` call.

The PyInstaller-frozen application does not have access to the python interpreter executable and must therefore use its embedded python interpreter, so any alternative python-based multiprocessing solution would also need to spawn its worker subprocesses using the program executable (`sys.executable`).

****FastAPI/Uvicorn with PyInstaller:****

When running FastAPI/Uvicorn with PyInstaller, it's recommended to call `multiprocessing.freeze_support()` before running the application and set `reload=False` and

workers=1.

****Common Issues:****

1. When using --onefile packaging with multiprocessing on Linux, issues can occur including FileNotFoundError with base_library.zip, and platform-conditional wrapping of freeze_support can lead to resource_tracker errors.

2. multiprocessing.freeze_support should be called before you instantiate your app and frame; otherwise, this instantiation will happen in every worker subprocess.

3. uvicorn workers can fail to start properly with PyInstaller, particularly when using multiple workers.

The search results show that the main recommendation is to use `reload=False` and `workers=1` when bundling with PyInstaller to avoid multiprocessing complications.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

20. assistant (2026-06-21T13:53:10.251Z)

I have comprehensive grounding now. Let me verify two final repo facts that are load-bearing for the recommendation: the actual server endpoint and worker subprocess invocation pattern (to confirm whether the engine spawns child Python processes by name, which matters for freezing).

21. user (2026-06-21T13:53:12.292Z)

Found 30 files limit: 30
backend\main.py
backend\config\models.py
backend\worker__main__.py
backend\compute\cloud_run.py
backend\config\startup.py
backend\config\presets.py
backend\pipeline\detectors\native_wasb_runner.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\detectors\wasb_infer.py
backend\worker__init__.py
backend\pipeline\stitcher\compiler.py
backend\eval\rally_seg_eval.py
backend\eval\fusion_compare.py
backend\eval\fusion_audio.py
backend\eval\ablation.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\detectors\wasb_runner.py
backend\pipeline\detectors\base.py
backend\eval\serve_contrast.py
backend\eval\distill_local.py
backend\eval\served_gate_a.py
backend\eval\fusion_golden.py
backend\utils\run_telemetry.py
backend\eval\calibrate_local.py
backend\pipeline\audio\hit_detector.py
backend\utils\video.py
backend\pipeline\validators\pts_continuity.py
backend\pipeline\validators\format.py
backend\pipeline\detectors__init__.py

22. user (2026-06-21T13:53:13.535Z)

735: uvicorn.run(app, host="127.0.0.1", port=args.port)

23. assistant (2026-06-21T13:53:17.067Z)

One important confirmation: `uvicorn.run(app, ...)` passes the app object directly (not an import string), with no `workers=``/reload=`` ? so it runs single-process, which is the PyInstaller-friendly pattern. Let me check how the WASB detector spawns its subprocess (WSL/conda), since that's the hardest packaging problem.

24. user (2026-06-21T13:53:19.119Z)

```
1: """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
6: 3. runs it in the WSL `wasb` conda env -> a Frame, Visibility, X, Y trajectory CSV,
15: import subprocess
21: from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22: from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin, wsl_tilde_quote
35:     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36:     conda_env: str = "wasb"
39:     wsl_stage_dir: str = "~/clips"
40:     timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
53:     distro = w.get("wsl_distro", "Ubuntu"),
55:     conda_sh = w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56:     conda_env = w.get("conda_env", "wasb"),
60:     wsl_stage_dir = w.get("wsl_stage_dir", "~/clips"),
74:     cmd = (f"conda activate {self.cfg.conda_env} && "
78:           res = self._wsl_bash(cmd)
80:           return False, "`wsl` not found ? Windows + WSL2 required."
81:     except subprocess.TimeoutExpired:
89:         src_mnt = to_wsl_mnt_path(video_win_path)
90:         reason = self.wsl_source_unreadable_reason(src_mnt)
94:         dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95:         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
{wsl_tilde_quote(dest)} && echo OK"
97:         res = self._wsl_bash(cmd)
98:     except subprocess.TimeoutExpired:
108:         src_mnt = to_wsl_mnt_path(_INFER_WIN)
110:         res = self._wsl_bash(cmd)
```

[Showing results with pagination = limit: 25]

25. user (2026-06-21T13:53:20.166Z)

```
36: import os
37: import shutil
38: import logging
39: import subprocess
40: from dataclasses import dataclass
41: from typing import Any, Dict, List, Optional, Tuple
43: from backend.pipeline.detectors.base import DetectorRunner
44: from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
46: from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
50: # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
54: # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution probe
56: _CUDA_PROBE_CODE = "import torch; print('torch', torch.__version__, 'cuda',
torch.cuda.is_available())"
117:     """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess). Any
failure
118:     (missing python / timeout / import error) is treated as 'no CUDA'."""
132:     """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs the
CUDA
142:     """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
158:     """Verify weights + repo present and the `wasb` python imports torch (and sees a GPU
on cuda)."""
177:         return False, "native torch healthcheck timed out."
180:         return False, f"{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}"
```



```
187:         return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
```

26. assistant (2026-06-21T13:53:25.944Z)

This confirms the key packaging-hostile fact: even the "native" WASB runner shells out to a separate Python interpreter (``self.cfg.python_bin``) running a copied ``wasb_infer.py`` in a separate torch env, and the WSL runner shells into ``conda activate wasb``. Both spawn external Python interpreters by path ? which a frozen single-binary cannot provide for the second torch env. I have everything I need.

Let me do one final confirmation search on ffmpeg/system-dep bundling and Mac notarization to round out the cross-platform signing claims.

27. user (2026-06-21T13:53:33.289Z)

Web search results for query: "macOS notarization Apple Developer ID python frozen app stapler PyInstaller 2025 requirement"

Links: [{"title": "Notarized app created with pyinsta? | Apple Developer Forums", "url": "https://developer.apple.com/forums/thread/695989"}, {"title": "Notarization | Apple Developer Forums", "url": "https://developer.apple.com/forums/tags/notarization?page=5"}, {"title": "Codesigning and Notarization of Py? | Apple Developer Forums", "url": "https://developer.apple.com/forums/thread/744471"}, {"title": "Notarized software still gets labe? | Apple Developer Forums", "url": "https://developer.apple.com/forums/thread/132908"}, {"title": "OS X Code Signing Pyinstaller.md · GitHub", "url": "https://gist.github.com/txoof/0636835d3cc65245c6288b2374799c43"}, {"title": "Pyinstaller exe fails when signed following apple notarization process · Issue #7937 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/7937"}, {"title": "Hardened Runtime Code Signing for the macOS .app bundle", "url": "https://groups.google.com/g/pyinstaller/c/csk5zLIwbmw"}, {"title": "Notarization of app built with Pyt? | Apple Developer Forums", "url": "https://developer.apple.com/forums/thread/691986"}]

Based on the search results, here's information about macOS notarization with PyInstaller and related tools:

Key Requirements for macOS Notarization with PyInstaller

****Code Signing with Hardened Runtime:**** To get notarized, one must "Enable the Hardened Runtime capability for your app and command line targets." This typically involves using the ``-o runtime`` flag when code signing.

****Entitlements for PyInstaller Apps:**** PyInstaller-built binaries require specific entitlements including ``com.apple.security.cs.allow-jit``, ``com.apple.security.cs.allow-unsigned-executable-memory``, and ``com.apple.security.cs.disable-library-validation``.

****Notarization and Stapling Process:**** The standard workflow involves submitting to notarization using ``xcrun notarytool --vvv submit --wait --keychain-profile`` and then stapling the receipt using ``xcrun stapler staple``.

****Modern Tools (2025):**** There is an excellent updated solution as of February 2025 using modern tools and an updated workflow, with the older ``altool`` being deprecated and requiring migration to ``notarytool``, with an updated solution available as of February 2025.

****Signing Complexity:**** One important consideration is that using the ``--deep`` flag is not recommended, and instead you need to sign each code item separately, from the inside out.

The search results show that developers have been successfully notarizing PyInstaller applications, though the process can be complex due to nested code signing requirements and proper entitlements configuration.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

28. assistant (2026-06-21T13:53:35.928Z)

I have all the grounding I need. Let me compile the structured research output.

29. user (2026-06-21T13:55:06.631Z)

Output does not match required schema: root: must have required property 'fit_for_this_app'

30. assistant (2026-06-21T13:55:09.524Z)

The `fit\_for\_this\_app` parameter needs to be in JSON string format. Let me resubmit.

31. user (2026-06-21T13:56:10.248Z)

Output does not match required schema: root: must have required property 'fit_for_this_app'

32. assistant (2026-06-21T13:56:13.110Z)

The `recommendation` and `fit\_for\_this\_app` need to be `` tags, not `>`. Let me correct the tag names.

33. user (2026-06-21T13:57:08.323Z)

Structured output provided successfully

34. assistant (2026-06-21T13:57:21.429Z)

Research complete. Structured output delivered.

Verdict: Freezing is ****VIABLE but tiered****, and should target a CLI/server appliance (per owner decision D10), not a GUI desktop app.

- ****Gemini-only / no-torch tier ? viable now, low risk.**** PyInstaller in ****onedir**** mode (not onefile) freezes the fastapi/uvicorn/opencv-headless/numpy/google-genai stack cleanly. The engine is already freeze-friendly: `backend/main.py:735` runs single-process uvicorn with the app object on 127.0.0.1 (no workers/reload), and cloud SDKs import lazily in `backend/storage/\*`.
- ****Full local-model (WASB) tier ? partially viable, defer.**** The real blocker is not the freezer but the ****dual-torch-env**** design: WASB runs in a separate torch-1.11 conda/WSL interpreter the engine shells out to (`wasb\_runner.py:74`, `native\_wasb\_runner.py:117`). No freezer bundles two interpreters in one process ? it needs either a second sidecar binary (~1GB+, double signing surface) or the strategically right path: finish the de-WSL native runner and ****export WASB to ONNX****, shipping one onnxruntime instead of torch.
- ****Tool pick:**** PyInstaller (onedir) default; Nuitka (Apache-2.0, built-in torch/onnxruntime pruning) as fallback once ONNX lands. Reject cx_Freeze (documented torch DLL pain) and Briefcase (explicitly weak on C-extension/ML deps).
- ****License:**** both PyInstaller (GPL-2.0 + bootloader exception) and Nuitka (Apache-2.0 core) let us ship MIT binaries without inheriting their license ? guardrail-clean.
- ****Separate non-trivial workstreams regardless of tool:**** ffmpeg as side-by-side LGPL binaries (D4, never GPL libx264), per-OS code-signing/notarization, and AV false-positive mitigation (onedir + signing).

Key files grounding this: `backend/main.py:735`, `backend/pipeline/detectors/wasb\_runner.py:74`, `backend/pipeline/detectors/native\_wasb\_runner.py:117`, `docs/DESKTOP\_APP\_PLAN.md:64`, and `C:/Users/avidu/Projects/khelsutra/docs/LOCAL\_APPLIANCE\_ARCHITECTURE.md:39`.